Chapter 3

Training Data Collection, Setup and Evaluation

3.1 Introduction: Why Training Data Is the Bottleneck

In many organizations, modeling gets most of the attention. In recommender systems, however, data quality and collection practices often have the largest impact on performance. You can implement the most sophisticated ranking model such as a multi-task neural network with cross-attention, learned embeddings, etc. and it will still underperform a simple XGBoost model if your training data is poorly collected, incorrectly labeled, biased by the previous system or otherwise suffers from a myriad of other problems like train-serve skew. This is not a hypothetical warning; it is the lived experience of nearly every ML team that has built production recommender systems.

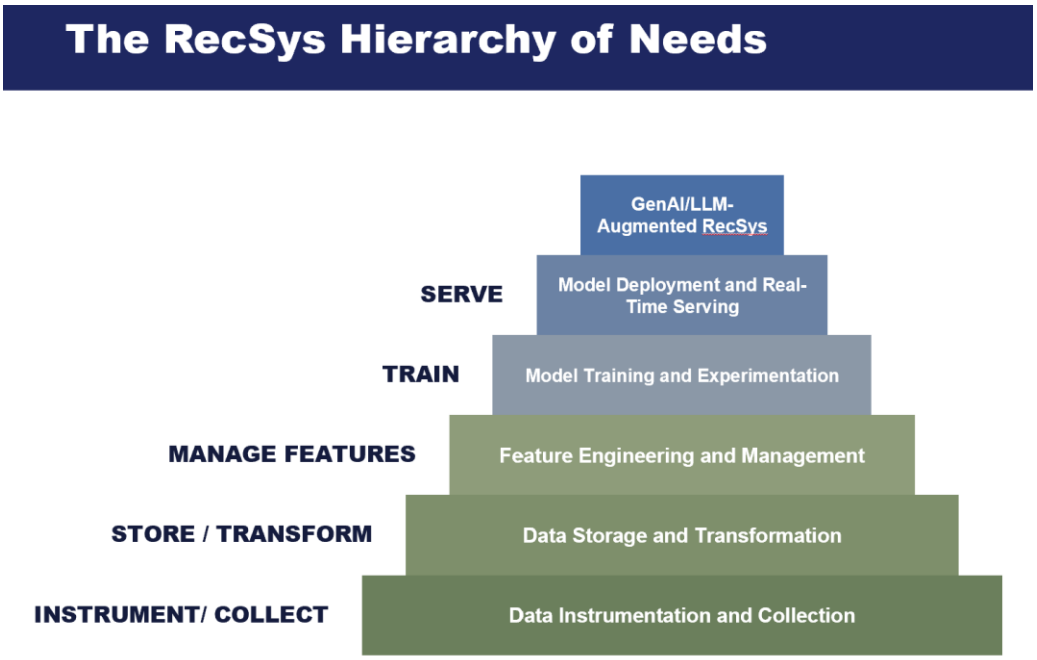

Figure 3.1: The RecSys Hierarchy of Needs, adapted from Monica Rogati's "AI Hierarchy of Needs" (Rogati, 2017). From bottom to top: (1) Data Instrumentation and Collection: clickstream, event logging, raw event sources; (2) Data Storage and Transformation: data warehouse, ETL/ELT pipelines, schema evolution, basic data-quality checks; (3) Feature Engineering and Management: offline and online feature stores, feature pipelines, point-in-time correctness, train-serve consistency; (4) Model Training and Experimentation: training pipelines, hyperparameter tracking, model registry, offline evaluation; (5) Model Deployment and Real-Time Serving: inference pipelines, monitoring, online experimentation, retraining loops. At the peak sits a frontier layer of Generative / LLM-Augmented RecSys (semantic IDs, generative retrieval, LLM-augmented features, LLM-as-recommender).

Before you can train a model, you need features. Before you can have features, you need data pipelines that generate them. Before you can have pipelines, you need reliable data instrumentation. This dependency chain is one of the most underappreciated realities in production ML, and it is especially acute in recommender systems, where every layer below the model determines whether the model's predictions can be trusted, served, and improved over time. Drawing on Monica Rogati's “AI Hierarchy of Needs” (Rogati, 2017), which she used to point out that most organisations skipping straight to AI had not yet mastered the layers beneath it, this chapter introduces a recsys-specific adaptation of the same idea.

Each upper layer is only as solid as the layers beneath it: organizations that jump straight to model training (layer 4) without investing in data quality, feature engineering, and storage (layers 2 and 3) will build models that are difficult to reproduce, impossible to serve consistently, and painful to maintain. In recommender systems, the consequences of inverting this hierarchy show up as a recurring set of failure modes: train-serve skew, where the same feature is computed differently offline and online; feedback loops, where the model's own outputs poison the next training run; position bias, which silently inflates offline metrics; and cold-start failures that production traffic surfaces but offline evaluation hides. Each is traceable to a missing or weak layer below the model.

The hierarchy is best read as a maturity diagnostic, not a syllabus. It’s a framework that can be used to communicate to engineering leadership on areas of investment to unlock the most leverage.

This chapter focuses on some layers in the pyramid above so that the ML Engineer can more confidently do the following: collect training data, turn raw user behavior into useful labels, identify and mitigate the biases that are endemic to RecSys data, and evaluate models before they reach production.

By the end of this chapter, you will be able to:

- Design a logging and instrumentation strategy that captures the signals your models need
- Construct training labels from implicit feedback while understanding their limitations
- Identify and mitigate common data quality issues including position bias, presentation bias, and feedback loops
- Implement temporal data splitting strategies that prevent data leakage
- Choose and compute appropriate offline evaluation metrics

3.2 Logging and Instrumentation

The quality of your recommender system is bounded by the quality of your data collection. This section covers the logging infrastructure that feeds your training pipeline.

3.2.1 Impression Logging: Capturing What Was Shown

Note that these clickstream logs captured from the customer-facing website/app are usually under the ownership of Data Engineering/analytics + the individual frontend software engineering teams, not the ML Engineering teams. However, despite being a different domain, it is important for ML Engineers to atleast have a rudimentary understanding of the attributes, without which the training pipeline logic can have serious bugs. The single most important data collection decision in RecSys is whether the organization correctly logs impressions on the set of items that were shown to a user, not just the items they clicked on.

Without impression logs, your training data consists only of positive examples (items that were clicked). This creates a fundamental problem: the model has no information about items that were shown but not clicked. Were they irrelevant? Or were they relevant but the user did not scroll far enough to see them? Without this information, you cannot construct reliable negative examples.

Many organizations, especially those that did not originally build their systems with ML in mind, have incomplete impression logging. Common failure modes include:

- Only logging clicks, not the full set of items displayed
- Logging impressions at the page level but not at the item level (knowing a user saw a page but not which specific items were visible)
- Inconsistent logging across platforms (web vs. mobile vs. email)
- Missing session identifiers that would link impressions to subsequent actions

If your organization currently lacks true impression logging, getting it implemented should be your highest priority, ahead of any model improvement work. Everything else in this chapter assumes you have it. Companies with customer-facing applications generally will atleast have page_loads: which records the full set of items returned by the server, rather than the subset of items actually visible to the customer. If you are not sure if your organization logs impressions as well in addition to page_loads, rather than making assumptions, it will be worthwhile exercise to sit down with the analytics team. In a Search example, mis-interpreting page_load events as impressions would inflate how many ‘negative’ samples there are per query. A page_load might have 100s of items while in fact, the user only saw a handful of items on their viewport.

3.2.2 Signals Schema Design

In the case of Search/Information Retrieval domain, it is useful to demarcate two broad approaches to solving for relevance: semantic relevance vs customer signals-based relevance. Approaches based on 'Semantic' relevance makes use of the content of the documents to surface what is relevant. For example, in an ecommerce domain, this might be the product title, price, category, reviews, etc. On the other hand, the signals-based relevance approach is driven by what did other customers who had a similar intent, action upon (either directly by boosting results that are popular for that search term or indirectly through learned models).

Each approach has its own strength and weaknesses. The latter approach will struggle in cold start situations when historical data is sparse. It will also amplify popularity bias in the absence of mitigation strategies. The former approach though initially appearing to be objective, can suffer from biases baked into the system – for example if you using an open-source language model to determine what is relevant, it is likely going off of the notion of relevance that its model developers used or from the dataset such as the MS Marco dataset. If your domain is radically different from the pre-training dataset/task, then you run the risk of the prioritizing attributes from your content metadata that your users don't find relevant.

In the case of Recsys/personalization, customer signals-based approaches are predominant because customer intent is often implicit in the latent[1] vector representation of the user journey, unlike search where the user explicitly types in a query. Item content is still used as features in recsys models but is not the primary means of understanding what is relevant, unless the use case is much narrower such as cold start recommendations from the product detail page in a retail ecommerce use case.

The key belief in the Recsys/IR community that underpins a lot of supervised Recsys ML is the idea of the feedback loop: user behavior signals when properly collected and aggregated, will reflect the collective intelligence of the user base[2].

[1] The author has built offline systems which convert raw clickstream events into a customer journey summary by feeding into an LLM. Using this summarized journey as the 'search query' in a simple semantic search achieves remarkably good intuitive results. It remains to be seen if similar approaches will be widely adopted for realtime inference at scale if LLM costs & latency drop 1-2 orders of magnitude.
[2] Grainger et al articulate this with 'Reflected Intelligence' in their excellent book "AI-Powered Search".

To capture this intelligence, you need a well-designed events schema. At minimum, each logged event in the frontend should include:

- **Event ID:** A unique identifier for this event
- **User ID:** An identifier for the user (or anonymous session ID)
- **Item ID:** The item that was interacted with
- **Event type:** What happened (impression, click, add-to-cart, purchase, like, skip, dwell, etc.)
- **Timestamp:** When the event occurred (with timezone)
- **Context:** The surface where this happened (homepage, search results, email, notification)
- **Position:** Where in the ranked list the item appeared
- **Session ID:** Which user session this belongs to
- **Request ID:** Which recommendation request generated this impression (links impressions to the model that generated them)

The request_id is often overlooked but is critical. It links a set of impressions to the specific recommendation request that generated them. Without it, you cannot reconstruct which items were shown together, which makes it impossible to implement pairwise or listwise training formulations.

Author's Note*: if this is your first project where you are working with a Data team that owns the raw clickstream events, it is often tempting for the ML teams to ask too much of them. The author has seen junior ML Engineers often demand features like "user_trailing_12months_purchase_amount" as part of the clickstream event logs. Logging features that can be derived easily in your SQL query after the fact is a bad idea for multiple reasons include a) logging more attributes can slow[3] down the website and b) its architecturally the wrong area. In chapter 4, we will get into more details on some common patterns for where features need to be computed.*

3.2.3 Coordinating with Frontend Teams for Richer Events

A common gap in many organizations is that clickstream collection is designed by frontend or analytics teams without input from the ML team. The result is that the richest interaction signals, the ones that happen before a click, are never captured.

[3] *Latency detriment harms business metrics that often can't be compensated by from increased accuracy. This is a classic speed vs accuracy tradeoff manifesting across different areas of your system.*

Consider an e-commerce product listing page. The standard analytics setup logs page views and clicks. But users exhibit many other meaningful behaviors:

- Hovering over a product image (interest signal)
- Scrolling past a product quickly vs. pausing (engagement signal)
- Using “quick view” or “read more” without a full click (soft interest)
- Adding to a comparison list or wishlist
- Zooming into an image
- Reading reviews on the listing page itself

These pre-click signals are valuable because they are much more abundant than clicks and can help disambiguate user intent. However, collecting them requires coordination between the ML team and the frontend engineering team several months ahead of time before the model development process to give them enough of a headsup to incorporate in their dev roadmap as well as allow enough time for sufficient training volume of collected events to accumulate.

Author’s Note: *When building a RecSys model from scratch or iterating on existing Recsys models that only had a barebones feature list, understanding[4] the product deeply is one of the highest ROI activities. Working sessions with your Product Manager, frontend engineering counterpart and/or analytics lead can lead to insights like ‘customers are actually using the Add-to-cart button as a way to save multiple items instead of the compare button’ . Which can then be used to generate powerful feature engineering ideas like “frequency of add-to-cart additions in last 24 hours*

3.2.4 Cross-Device User Tracking

Users increasingly interact with products across multiple devices. For example, browsing on mobile during a commute, and then purchasing on a desktop computer once home. Without cross-device identity resolution, these appear as separate users, fragmenting your behavioral data, and ultimately reducing the length of user sequence history you can feed into your models. Approaches to cross-device tracking include:

- **Deterministic matching:** Users who are logged into the same account across devices. This is the gold standard but requires a login.

[4] *Gaining a deep ‘product sense’ for your company’s product is a key milestone on the road to becoming an excellent Recsys ML Engineer.*

- **Probabilistic matching:** Using device fingerprints, IP addresses, and behavioral patterns to infer that two anonymous sessions belong to the same user. Less reliable but works for non-logged-in users.
- **Session-level fallback:** When you cannot resolve identity, fall back to session-level features. Session-based models (Chapter 11) can still provide personalization within a single visit.

The privacy landscape (GDPR, CCPA, cookie deprecation, etc.) is making cross-device tracking increasingly tricky. This is one reason why first-party data (logged-in user behavior) is becoming more valuable and why content-based approaches that do not rely on user history are maintaining their relevance. We won't cover this in our book because implementing this tracking is often beyond the domain of ML teams (and usually falls under some analytics or data platform teams).

3.3 Label Engineering

Turning raw behavioral signals into training labels is both an art and an engineering challenge. The fundamental difficulty is that we want to predict user preference, but we only observe user behavior. Behavior is a noisy, biased proxy for preference.

3.3.1 The Ambiguity of Implicit Feedback

Consider a video streaming service. A user clicks on a video, watches for 2 minutes, and then stops. For the purposes of supervised ML training data: is this record to be labelled[5] as a positive example or a negative one? The answer depends on the movie's length, the user's typical behavior, and the reason they stopped:

- If the video is a reel about 3 minutes long, 2 minutes of watch time is a strong positive signal.
- If the video is a recorded lecture about 120 minutes long, watching only 2 minutes suggests the user was not engaged
- If the user's phone rang and they intended to come back, it is a false negative.
- If the user fell asleep with autoplay running, even a full 120 minutes watch might be a false positive.

This ambiguity is not a niche problem, it is the fundamental challenge of implicit feedback. Every implicit signal (clicks, dwell time, purchases, skips) has its own failure modes:

[5] A more rigorous treatment of how to interpret user clicks in the context of Search/IR is available in the short book "Click Models for Web Search" available at https://clickmodels.weebly.com/the-book.html

Clicks can be driven by curiosity, misleading thumbnails, or accidental taps. A clickbait video might generate high CTR but low satisfaction.

Dwell time can be inflated by distraction (leaving a tab open) or deflated by efficient browsing (finding what you need quickly).

Purchases are a strong signal but sparse and delayed. Most browsing sessions do not end in a purchase. A customer looking to buy a high value item like Washer-dryer might end up comparing the options for multiple days/weeks before making a decision.

Skips are a relatively clean negative signal but are only available in sequential consumption contexts (music, video playlists).

3.3.2 Combining Multiple Signals

The solution is rarely to rely on a single implicit signal. Instead, mature Recsys teams build training pipelines that combine multiple signals to construct a more robust label:

- **Composite labels:** click AND dwell time above threshold → positive; click but immediate bounce → negative; no click → negative
- **Weighted labels:** assign different weights based on ordinal ranking of signal strength (purchase > add-to-cart > click > impression).
- **Multi-task labels:** instead of engineering a single composite label, predict each signal separately and combine predictions with a value function (the approach we use in Chapters 7-8)

The multi-task approach is increasingly preferred in production systems because it avoids the need to engineer a “perfect” composite label. Instead, you let the model learn the relationship between different engagement signals, and the finaly re-ranker or value function (Chapter 8) handles the combination based on business priorities.

3.3.3 Time Decay for Popularity and Recency Signals

Not all interactions are equally relevant over time. Most signals lose relevance with age. A click from yesterday is a stronger statement of current intent than a click from three months ago, and treating both equally would let stale behavior overwhelm the freshest evidence. Time decay functions discount older signals, giving more weight to recent behavior. A common formulation is a simple inverse function such as (the ϵ term avoids divide-by-zero when signal_age = 0):

$$\text{weight} = \frac{1}{\text{signal_age} + \epsilon}$$

The standard way to encode this in a feature pipeline is exponential decay, written in half-life form so the parameter stays interpretable:

$$weight = starting_weight \cdot e^{-\lambda \cdot signal_age}$$

Where,

1. *starting_weight* reflects the signal type (e.g., 1 for a click, 5 for a purchase),
2. $\lambda = \frac{\ln 2}{\text{half_life}}$, the numerator is the natural log of 2, the denominator half_life determines how quickly signals lose relevance.
3. *signal_age* is how old the interaction is.

Why exponential is the default for click decay: It has the memoryless property (the relative drop[6] over the next day is the same regardless of how old the signal already is), it's smooth with no abrupt cutoff, and the total weighted contribution of an infinite history is bounded. Those three properties together make it well-behaved for streaming aggregations and online updates,

Time-decayed popularity features and time-decayed user preference features are among the most enhancements to a feature set. Many teams maintain different half-lives per signal type so that strong signals like purchases decay slowly while noisier signals like view-throughs decay quickly.

Choice of half-life is domain-dependent: news feeds typically use hours, short-form video uses days, e-commerce browsing uses days to weeks, and purchase or subscription signals use months.

- **News recommendations:** half-life of hours (content goes stale quickly)
- **E-commerce:** half-life of days to weeks (purchase intent evolves)
- **Music streaming:** half-life of weeks to months (musical taste is more stable)

Other functional forms exist (linear, power-law, and fixed-window), each with different long-tail behavior, but exponential decay is the default in most production click pipelines because of its memoryless property and its compatibility with streaming updates.

Author's note: This is another example of the demarcation between Science vs. Engineering blurring in Recsys – feature engineering choices are co-determined based on tradeoffs in computational complexity, rather than purely based on functional usefulness. If your org has separate ML Science vs Data Engineering/ML Engineering

[6] This functional form shows up in a lot of natural phenomena including radio isotope decay, social sciences, etc.

roles, make sure you are tightly in synch with your progress rather than waiting till the last minute before prod to find out if something is feasible.

3.3.4 Position Bias and How to Mitigate It

Position bias is one of the most well-documented biases in RecSys and search systems: users are far more likely to interact with items at the top of a ranked list, regardless of those items' intrinsic relevance. An item in position 1 might receive 10x more clicks than an equally relevant item in position 5, simply because users scan from top to bottom and often stop before reaching lower positions.

This creates a pernicious feedback loop: the current model's top-ranked items get more clicks → those items appear more relevant in training data → the next model ranks them even higher. Over time, this can cause your model to simply memorize the previous ranking rather than learning true relevance.

Mitigation[7] strategies include:

- **Position as a feature:** Include the position at which an item was shown as a feature during training. At inference time, set this feature to a constant value (e.g., 1) so the model predicts engagement as if every item were in the top position. This debiases the predictions and is recommended as Rule #36 in Google's Rules of Machine Learning (Zinkevich, 2017).
- **Inverse propensity weighting (Joachims, Swaminathan, & Schnabel, 2017):** Weight the training examples by the inverse of their examination probability (estimated from position). Items in lower positions that were still clicked get higher weight.
- **Click models:** Probabilistic models (Chuklin, Markov, & de Rijke, 2015)[8] that separate the examination probability (*did the user see the item?*) from the relevance probability (*given they saw it, did they engage?*).
- **Randomization:** Serve a small fraction of traffic with randomized rankings and use that unbiased data for training or evaluation. Expensive in terms of user experience since it will likely cause detriment to business metrics but provides clean signal.
- **Hyperparameter within the algorithm**: In some algorithmic implementations, position bias correction is built in as a parameter. XGBoost's Learning to Rank

[7] Eugene Yan has an excellent blog post that describe these techniques in more detail. https://eugeneyan.com/writing/position-bias/

[8] The Simplified Dependent Cascade Model (SDBN) from the "Click Models for Web Search" literature (Chuklin et al., 2015) provides a more advanced framework.

(LTR) supports debiasing by setting the `lambdarank_unbiased=True`[9] for the rank:ndcg objective (XGBoost Developers, n.d.).

3.3.5 Presentation Bias

Beyond position bias, there is a subtler issue: presentation bias. Your training data only contains examples of items that your current system chose to show. Items that were never presented to users, because the retrieval model did not surface them or the previous ranking model scored them low, have no interaction data at all.

This means your training data is drawn from a biased distribution: it over-represents items that the old model preferred and under-represents items that the old model ignored. The fact that the older model did not surface doesn't necessarily mean that if they were surfaced the users would have found it less relevant. This often exacerbates cold start issues for new items.

Mitigation strategies:

- **Exploration traffic:** Reserve 1-5% of traffic for serving random or diversified recommendations. This generates training data for items that would otherwise never be shown.
- **Counterfactual evaluation:** Use techniques like inverse propensity scoring to estimate what would have happened under a different recommendation policy.
- **Cold-start injection:** Periodically inject new or under-exposed items into the recommendation stream, even if the model does not rank them highly.

3.4 Common Data Quality Issues

Beyond the labeling challenges discussed above, several data quality issues are endemic to RecSys datasets.

3.4.1 Feedback Loops

We touched on feedback loops in Chapter 1. Here, let us be more specific about how they manifest in training data.

When the serving policy determines exposure and exposure determines the training logs, you get a feedback loop. Under-expose a category and you observe fewer

[9] While its not the goal of the book to deep dive into the Pythonic implementations of specific algorithms, XGBoost gives very good results out of the box that its worth a special mention. Additionally, most ML Scientists coming from other domains likely will already be familiar with using XGBoost.

interactions, not necessarily because users dislike it, but because they didn't see it. Trained naively, the next model downranks the category again.

The result is shrinking catalog coverage and a more repetitive experience.

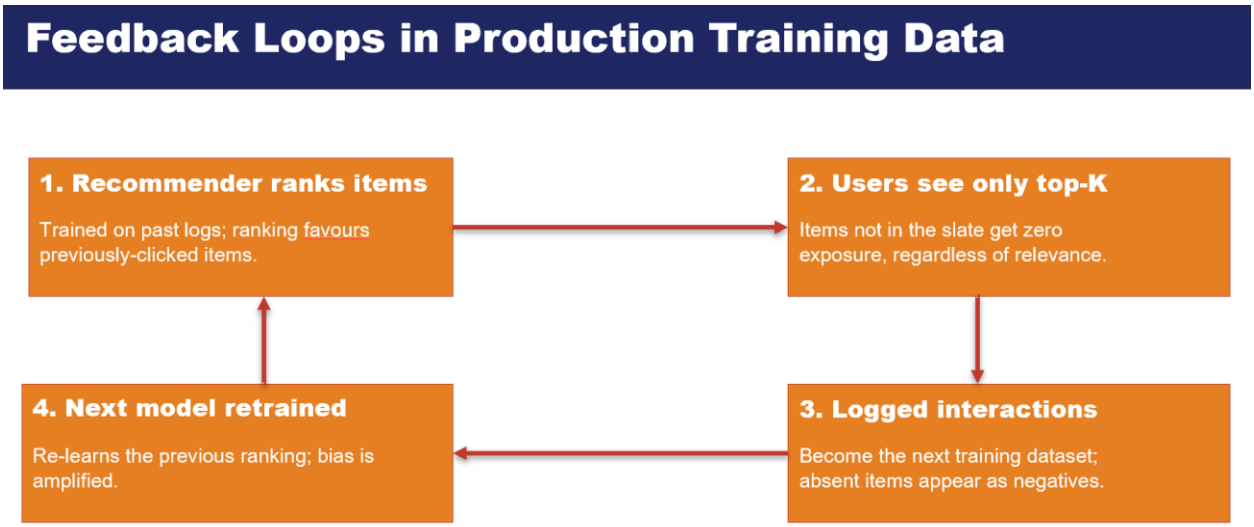

Figure 3.2: The feedback loop in production recommender training data. The current ranker determines exposure; users can only interact with exposed items; those interactions become the next training set; the next ranker repeats and amplifies the previous bias. The result over time is shrinking catalog coverage, an increasingly repetitive user experience, and offline metrics that look healthy because the model is learning to predict its own past behavior.

Some mitigation strategies:

- **Diversity constraints:** Force the recommendation system to show items from across the catalog, not just the model's top predictions.
- **Regular retraining with fresh data:** The model should see recent interaction patterns, not just historical biases.
- **Offline–online correlation tracking:** Track whether offline metric lifts (e.g., NDCG on a temporally held-out set) translate into online lifts (CTR, engagement). Persistent divergence usually signals an evaluation mismatch or logging/serving bias (including policy feedback effects) and should trigger investigation.

3.4.2 Cold-Start for Users and Items

'Cold-start' is the challenge of serving your users under conditions of limited data. As with many other engineering terms, this is an overloaded term with the meaning completely changing depending on the context in which it is being discussed. A couple of the cold start variants are described below:

User cold-start: This is often used to describe new users with no features (typically user profile or interaction history). Approaches to mitigate impact to accuracy include:

- Content-based recommendations using item metadata
- Popularity-based fallback
- Onboarding flows that collect explicit[10] user preferences
- Contextual features (time, location, device) that provide weak signals even without history

Author's note: in the author's experience, if your application is fairly mature, customers are visiting frequently **and** there is a fairly robust way for cross-device user authentication, then it is less of a problem. On the other hand, if you have some combination of

a) New app launch or
b) Rapidly growing customer base or
c) Large number of unrecognized visitors with limited ability to resolve their identities without them explicitly logging in,

Then the impact to the model's accuracy can be quite severe.

During offline evaluation, there are a few analysis you could perform to understand how much detriment user cold start is having on metrics.

a) Slice by recognized vs unrecognized customers
b) Slice by customers with no user profile and historical user-item interactions vs those who have it.

If the model is trained well, it often is the case that even with the user features being null, the model predictions roughly match the rank order given by a simple popularity heuristic. This is a good check for model robustness. When all the user features are 0, if the results look terrible, its worth revisiting the architecture to see if it makes sense to use a fallback popularity-based ranking or some other heuristic for cold start visitors.

[10] Be wary of divergence between what users state they want (explicit) vs what they actually do aka revealed preferences (implicit). As mentioned in an earlier chapter, the industry has long since moved on from relying on explicit user preferences exclusively.

Item cold-start: New items with no interaction data. Approaches include:

- Content-based retrieval using item metadata and embeddings (Chapter 10 explores how pre-trained text encoders like SBERT can embed new items without any interaction data)
- Explore-exploit strategies that deliberately expose new items to users (Chapter 16)
- Transfer learning from similar items

The severity of cold-start depends on your domain as well as whether you need to worry more about user cold start or item cold start. Platforms with high user and item turnover (e.g., job boards, real estate, flash sales) face both. Platforms with rapidly growing user base will face user cold start.

***Author's note**: The opposite case where teams over-play the impact of popularity bias and underplay the benefit of using clickstream labels in supervised ML has also been seen by the author. In one of the projects, stakeholders initially prohibited the use of clickstream feedback signals in supervised ranking models, fearing popularity bias from the existing legacy engine. This design choice led to the predictable outcome of the newly built system showing semantically correct but highly obscure results[11].*

If your org is just starting out building AI-based search/recsys architectures to beat legacy systems, first get to parity and demonstrate that your business metrics are responsive to your new system. Keep the system simple and follow existing industry patterns. This often means using clickstream signals in supervised ML models somewhere in your retrieval and/or ranking models.

This will require negotiating with stakeholders to temporarily tolerate[12] cold start issues/popularity bias until the new system is stable. Actual production systems at scale have way more complexity beyond the ML layer: it is not uncommon during initial iterations of the overall system for teams to realize that nothing they do actually moves the needle for business metrics!

3.4.3 Data Quality Checks Before Training

Back in the 'old' days, Data Scientists would first spend several weeks doing extensive exploratory data analysis to get an intuition for the data. Given the wide adoption of the AI coding tools, many of these steps can be automatically within a few minutes with

[11] In that particular project, there were a few other technical & non-technical issues including extremely slow end-to-end latency, highly restrictive business filters applied on the final results, and a mis-match between the metric that the system was optimized for vs what the business actually cared about.
[12] Manage expectations with the business stakeholders if an X% percentage drop in accuracy can be traded of for a Y% improvement against some cold start metrics.

simple prompts. Nevertheless, it is worth mentioning some critical things to check drawing on best practices from the ML systems community:

Label quality checks:

- What is the positive rate (fraction of positive labels)? Is it reasonable for your domain? E-commerce CTR is typically 1-5%; music listen rate might be in double digits. Dividing clicks by impressions across any time window for a specific user should ideally not[13] give you more than 100%, otherwise any models that need to predict calibrated CTR will be impacted
- Are there time periods with anomalous label rates?
- Is the label delay (time between impression and label assignment) consistent? If the click is recorded as taking place before the impression, this may indicate a bug.

Volume checks:

- Is there enough data for reliable model training for business metric of interest? Rule of thumb[14]: at least 10x the number of features in positive examples)
- Is the data balanced enough across user segments and item categories?

Completeness checks:

- Are there missing values in critical features? What is the missing rate by feature?
- What % of the users have no user-interactions in the training window?
- Can the labels be created for all records? Does the clickstream log have gaps? (e.g., corrupted/missing impressions for a specific platform or time range)?

Consistency checks:

- Do feature distributions match between training and serving data?
- Are there sudden distributional shifts that suggest a logging bug?
- Do categorical features have unexpected new categories?

These categories of checks are not exhaustive but rather a starting point to trigger further investigations & conversations with your data partners.

[13] The author has seen CTR exceeding 100%, due to a combination of mis-firing clicks and under-counted impressions.
[14] this 'events-per-variable' guideline originates from logistic regression literature, see Peduzzi et al., 1996, and modern deep models with many parameters typically need substantially more.

3.5 Train-Evaluation Splitting

How you split your data into training and evaluation sets is critical, and RecSys has specific requirements that differ from standard ML practice.

3.5.1 Temporal Splits and Data Leakage Prevention

If you are training your model on user-level[15] interactions**: Never use random splits. Always split by time.**

Your collected clickstream events should have timestamps for each user interaction. In a temporal split, all training data comes from before a cutoff date and all evaluation data comes from after it. This mimics the real-world scenario where you train on historical data and predict future interactions. Random splits violate temporal causality by mixing the same users trajectories across training and evaluation, effectively letting the model learn from information that would not be available at prediction time. The result is information leakage that inflates offline metrics and gives a false sense of model quality. If the split is leaky, even a model that largely memorizes user-item interactions can score well offline, making overfitting much harder to detect.

Note that this is a subtly different issue compared to point-in-time feature correctness: you can have perfectly point-in-time-correct features, but still get optimistic evaluation if the train/eval split mixes time and allows the model to learn associations that would only be observable in the future.

In Chapter 6, we implement a Global Temporal Split (GTS) on the Yandex Yambda dataset, where we use interactions from the first T days for training and the remaining days for evaluation. This approach naturally handles the sequential nature of user behavior and prevents leakage. Additional leakage pitfalls to watch for:

- **Feature leakage:** Using features that encode information from the future (e.g., an item's total purchase count computed over the entire dataset, including the evaluation period).
- **Validation[16] set contamination:** Using the validation set to make feature engineering or architecture decisions and then evaluating on the same set. Reserve a true held-out test set that is never touched during development.

[15] Alternative is to aggregate at say a 'daily' level across users. This is not practical in most recsys cases, and is seen more commonly in search where you can aggregate impressions by QueryID. So all the user impressions and clicks for a query like 'gaming laptop' can be aggregated together. This will increase signal-to-noise ratio especially for long tail searches and makes the training data size more manageable.
[16] Another subtle error can creep up when you train a calibration model (e.g., isotonic regression) to convert ranking model scores into a calibrated probability (e.g. pCTR). Make sure to use separate validation sets between the two models.

3.5.2 Session-Based Grouping

When splitting data, ensure that entire user sessions stay together. If a user's session spans the split boundary, you risk having the same session's interactions in both training and evaluation sets.

Depending on your schema, this could mean grouping interactions by session_id and assigning entire sessions to either the training or evaluation set based on the session's start time.

3.5.3 Negative Sampling Strategies

Your evaluation set needs both positive and negative examples. If your clickstream events are setup correctly and your frontend displays more items for the users to see in a typical session compared to what they typically interact with, then you have less of a concern. Even then, augmenting your training labels with intelligently chosen negatives can increase model accuracy. There are several strategies that are often model-specific. We will be covering some of them in later chapters.

The choice of negative sampling strategy significantly affects your offline metrics. Models that look great with sampled negatives (which are often easy to distinguish from positives) may look much less impressive with impression-based negatives (which include items that were plausibly relevant but not clicked).

3.6 Offline Evaluation Metrics

Offline evaluation is how you compare models before committing to an online A/B test. The key principle: evaluate your model the way it will be used in production. The output from Recsys APIs are often (but not always) served as a ranked list of items to the user. This motivates understanding ranking metrics[17] in more detail.

3.6.1 Ranking Metrics

NDCG@k (Normalized Discounted Cumulative Gain):

This is the most common ranking metric and it measures ranking quality by giving more credit for relevant items near the top, and it supports graded relevance (e.g., 0–3), not just binary. NDCG@k assigns a *gain* to each result based on its relevance (often $2^{rel} - 1$) and *discounts* that gain by its rank position (commonly $1/\log_2(i+1)$), so highly

[17] Don't be afraid to tweak the formulas or invent your bespoke ranking metric for your use case!

relevant items contribute much more when they appear near the top. The score is then normalized by the ideal DCG (IDCG) to range from 0 to 1.

Let rel_ibe the relevance of the item at rank i(1-based).

DCG@k:

$$\mathrm{DCG@}k = \sum_{i=1}^{k} \frac{2^{rel_i} - 1}{\log_2(i+1)}$$

IDCG@k (ideal DCG@k): the DCG@k of the *perfectly sorted* ranking (relevances in descending order).

NDCG@k:

$$\mathrm{NDCG@}k = \frac{\mathrm{DCG@}k}{\mathrm{IDCG@}k}$$

It typically ranges from 0 to 1 (with 1 meaning the ranking matches the ideal ordering for the top k).

MRR (Mean Reciprocal Rank):

This is defined as the average of 1/rank of the first relevant item across all queries. Useful when you care **only** about the top result (e.g., “did we put a relevant item in the top 3?”). This is a binary metric, it doesn’t take into account ordinal ratings by your users.

Mean Reciprocal Rank (MRR) is defined as:

$$\mathrm{MRR} = \frac{1}{|\,Q\,|} \sum_{q \in Q} \frac{1}{\mathrm{rank}_q}$$

where rank_q is the 1-based rank of the first relevant item for query q(and is typically treated as ∞, contributing 0, if no relevant item is retrieved).

Hit@k: The fraction of queries where at least one relevant item appears in the top k. A simple, intuitive metric that is easy to communicate to non-technical stakeholders. If a query has multiple relevant items, Hit@k still caps out at 1 for that query (it does not reward finding more than one relevant item). It’s equivalent to averaging an indicator over queries.

The mathematical notation is:

$$\text{Hit@}k = \frac{1}{\mid Q \mid} \sum_{q \in Q} \mathbf{1}\{\exists\, i \in \text{TopK}(q) \cap \text{Rel}(q)\}$$

Where $\mid Q \mid$ in the denominator is the count of all queries in the evaluation set. The summation term is basically incrementing counts when there is at least one item i that is both in the top-k results for query q and relevant for q.

MAP (Mean Average Precision): This is the mean, over queries, of Average Precision (AP), where AP measures how well relevant items are ranked throughout the list (it rewards putting relevant items early, and rewards retrieving *multiple* relevant items, unlike Hit@k). Qbe the set of queries, $R_q$the set of relevant items for query q, and $\text{rel}_q(i) \in \{0,1\}$ indicate whether the item at rank i is relevant.

Precision@k:

$$P_q(k) = \frac{1}{k} \sum_{i=1}^{k} \text{rel}_q\,(i)$$

Average Precision (AP) for a query q:

$$\text{AP}(q) = \frac{1}{\mid R_q \mid} \sum_{i=1}^{N} P_q\,(i) \cdot \text{rel}_q(i)$$

(where N is the ranked list length; in practice you can use $N = k$to define $\text{AP@}k$.)

Mean Average Precision (MAP):

$$\text{MAP} = \frac{1}{\mid Q \mid} \sum_{q \in Q} \text{AP}\,(q)$$

If a query has **no relevant items** ($\mid R_q \mid = 0$), AP is typically defined as 0 or you can exclude that from your evaluation set (the latter is the safer choice).

3.6.2 Python Implementations of Ranking Metrics

These reference implementations match the formulas above. They use only the Python standard library and prioritize clarity over speed; production systems typically use vectorized NumPy or PyTorch versions for batch evaluation across thousands of queries.

NDCG@k:

```
import math

def dcg_at_k(relevances, k):
    return sum((2**rel - 1) / math.log2(i + 2)
               for i, rel in enumerate(relevances[:k]))

def ndcg_at_k(relevances, k):
    actual = dcg_at_k(relevances, k)
    ideal  = dcg_at_k(sorted(relevances, reverse=True), k)
    return actual / ideal if ideal > 0 else 0.0
```

MRR:

```
def reciprocal_rank(ranked_items, relevant_set):
    for rank, item in enumerate(ranked_items, start=1):
        if item in relevant_set:
            return 1.0 / rank
    return 0.0

def mrr(per_query_rankings, per_query_relevant):
    n = len(per_query_rankings)
    return sum(reciprocal_rank(r, s)
               for r, s in zip(per_query_rankings, per_query_relevant)) / n
```

Hit@k:

```
def hit_at_k(ranked_items, relevant_set, k):
    return 1 if any(item in relevant_set for item in ranked_items[:k]) else 0

def mean_hit_at_k(per_query_rankings, per_query_relevant, k):
    n = len(per_query_rankings)
    return sum(hit_at_k(r, s, k)
               for r, s in zip(per_query_rankings, per_query_relevant)) / n
```

Precision@k, Average Precision, and MAP:

```
def precision_at_k(ranked_items, relevant_set, k):
    top_k = ranked_items[:k]
    return sum(1 for item in top_k if item in relevant_set) / k

def average_precision(ranked_items, relevant_set):
    if not relevant_set:
        return 0.0
    score, hits = 0.0, 0
    for k, item in enumerate(ranked_items, start=1):
        if item in relevant_set:
            hits += 1
            score += hits / k
    return score / len(relevant_set)

def map_score(per_query_rankings, per_query_relevant):
    n = len(per_query_rankings)
    return sum(average_precision(r, s)
               for r, s in zip(per_query_rankings, per_query_relevant)) / n
```

3.6.3 Baselines: Random and Popularity Rankers

Always evaluate your model against at least two baselines:

Random ranker: Randomly shuffle your candidate set and evaluate against the holdout. Any reasonable model should substantially beat this. If your model barely outperforms random, something is fundamentally wrong with your data or labeling.

Popularity ranker: Ranks items by global popularity (interaction count). This is a surprisingly strong baseline: popular items are popular for a reason. If your personalized model cannot beat popularity, it suggests that either your personalization features are not informative, or your training data is too biased toward popular items. If you want to make it more sophisticated, try with different aggregation windows/time decay factors.

Even if it later turns out that your ML model handsomely beats the popularity baseline not only on aggregate but on all slices of interest (e.g., cold start users), its not wasted effort because

a) Popularity or random ranker can serve as a dummy model when setting up the CI/CD pipeline for your non-prod API

b) Popularity features are extremely useful in most ranking models

c) The percentage lift improvement over the baselines can be used as a guardrail when retraining new models.

In our Chapter 5 retrieval tutorials, the “last item browsed” heuristic serves as another powerful baseline. In Chapter 6, we carefully benchmark XGBoost against both random and popularity baselines on the Yambda dataset.

3.6.4 Beyond Accuracy: Diversity, Coverage, and Serendipity

Ranking accuracy alone does not make a good recommender system. Production systems also need to optimize for:

Diversity: Are the top-k items sufficiently different from each other? Measured by Intra-List Diversity (ILD): the average pairwise dissimilarity among recommended items. In Chapter 8, we compute ILD using item embedding distances.

Coverage: What fraction of the item catalog is ever recommended? Low coverage suggests the model is stuck recommending a small subset of popular items (likely due to presentation bias in the platform)

Serendipity: Are the recommendations surprising and valuable? This is hard to measure offline but is critical for user satisfaction. A system that only recommends

“safe” choices may score well on accuracy metrics but feel boring to users and can cause a longer term decline in the platform usage.

In Chapter 8, we track all three of these alongside NDCG as part of our value function and diversity optimization evaluation.

3.7 Summary

Training data is the foundation on which your entire recommender system rests. In this chapter, we covered:

- The critical importance of impression logging and well-designed event schemas
- How to coordinate with frontend teams to capture richer interaction signals
- The fundamental ambiguity of implicit feedback and strategies for constructing robust labels
- Time decay for popularity and recency signals
- Position bias and presentation bias — two endemic biases in RecSys data — and their mitigations
- Feedback loops, cold-start challenges, and data quality checks
- Temporal splitting strategies that prevent data leakage
- Offline evaluation metrics: NDCG, MRR, Hit@k, and beyond-accuracy metrics like diversity and coverage

In the next chapter, we turn to the infrastructure that makes all of this possible at scale: data pipelines, feature stores, and the engineering practices that prevent train-serve skew.

References

- Chuklin, A., Markov, I., & de Rijke, M. (2015). Click Models for Web Search. Synthesis Lectures on Information Concepts, Retrieval, and Services. Morgan & Claypool.
- Joachims, T., Granka, L., Pan, B., Hembrooke, H., & Gay, G. (2005). Accurately Interpreting Clickthrough Data as Implicit Feedback. SIGIR.
- Chen, M., et al. (2023). Bias and Debias in Recommender System: A Survey and Future Directions. ACM Transactions on Information Systems.
- Dowling, J. (2024). Building ML Systems with a Feature Store. O’Reilly Media.

- Rogati, M. (2017). The AI Hierarchy of Needs. Hacker Noon. https://hackernoon.com/the-ai-hierarchy-of-needs-18f111fcc007
- Grainger, T., Turnbull, D., & Irwin, M. (2024). AI-Powered Search. Manning Publications.
- Zinkevich, M. (2017). Rules of Machine Learning: Best Practices for ML Engineering. Google Developers. https://developers.google.com/machine-learning/guides/rules-of-ml
- Peduzzi, P., Concato, J., Kemper, E., Holford, T. R., & Feinstein, A. R. (1996). A simulation study of the number of events per variable in logistic regression analysis. Journal of Clinical Epidemiology, 49(12), 1373-1379.
- Yan, E. (2022). Position Bias in Recommender Systems. eugeneyan.com. https://eugeneyan.com/writing/position-bias/
- Joachims, T., Swaminathan, A., & Schnabel, T. (2017). Unbiased Learning-to-Rank with Biased Feedback. WSDM 2017. https://arxiv.org/abs/1608.04468
- XGBoost Developers. (n.d.). Learning to Rank: Position Bias. XGBoost Documentation. https://xgboost.readthedocs.io/en/latest/tutorials/learning_to_rank.html#position-bias